Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

VitalRoute: A Smart Multi-Specialty Hospital Management and Emergency Routing System

Team Name: *Team #*	*Team VitalRoute* *DSCPP-III-2026-T120*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Agarwal, Nakshatra – 2510370215* *2510370215@geu.ac.in* *Section: ML1* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Saini, Devesh – 2510370251* *2510370251@geu.ac.in* *Section: ML4* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Sharma, Vasav – 25110224030* *25110224030@geu.ac.in* *Section: E* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

A hospital has to answer a few questions quickly and correctly: which patient is seen first, whether a bed or an operation theatre is actually free, how the patient gets there, and what the stay finally costs. VitalRoute is a console simulator of those decisions. It is written in two language layers, a core engine in C that implements the data structures, and an object-oriented layer in C++ that models patients, staff, charges and bills on top of that engine.

Emergency arrivals are ranked by clinical severity in a priority queue instead of by arrival order. Each one passes through a shared central emergency room for stabilisation before being sent on to an intensive care bed, an operation theatre or a ward. Routine outpatients wait in an ordinary queue. Operation theatre slots repeat every day, so a circular queue manages them, and ambulance or transfer requests use a double-ended queue so that a genuinely urgent transfer can be placed at the front of the line.

The new capability in this phase is in-hospital navigation. The hospital layout is stored as a weighted graph whose vertices are corridors, junctions, lifts and rooms spread over several floors. Once the system has picked a destination bed, it runs Dijkstra's algorithm to find the shortest route, then translates that route into plain walking directions such as turn right at the nursing station, then take the lift to the third floor. An attendant who does not know the building can still follow it.

The remaining modules complete the patient journey. Visit history sits in a doubly linked list, hashing gives instant lookup by patient identifier, an AVL tree answers date-range queries, and file handling keeps every record between runs.

Updated Project Approach and Architecture

The layered architecture from Phase I has been extended rather than replaced. A console presentation layer sits above the application logic, which in turn rests on a shared data-structure engine written in C and on a file-based persistence layer.

What changed is the application layer, which now holds four modules instead of three. Patient and Staff Management, Emergency Triage and ER Routing, and Consultation and Billing were carried over from Phase I. Navigation is new.

The routing work has also been split into two separate stages, because the two stages answer different questions. The first decides which resource the patient should occupy. Candidates are filtered by the required type, so a general ward bed is never offered to a patient who needs intensive care, and the nearest valid candidate is then chosen by running Dijkstra's algorithm over the hospital graph. The second stage decides how to reach that bed, and converts the vertex sequence returned by the search into instructions.

Supporting the second stage meant extending the graph itself. Every vertex now stores a floor number and a pair of plan coordinates. Every edge stores a type, which is either a corridor, a lift or a stairway. Walking directions come from comparing the bearing of one segment with the bearing of the next, then classifying the difference as a left turn, a right turn, a slight turn or straight ahead. Lift and stairway edges are handled separately and produce a floor-change instruction rather than a turn.

The project uses no external libraries, no database engine and no network communication. Only the standard C library and the C++ Standard Template Library are involved, every core structure is written by hand, and the build is done with GCC and G++.

Tasks Completed

Task Completed	Team Member
Background study of emergency department practice, including five-level triage scales, and the feature freeze for Phase II	Nakshatra Agarwal
Hospital layout modelled as a weighted multi-floor graph: vertex schema with floor and coordinates, edge types for corridor, lift and stairway	Nakshatra Agarwal
Direction-synthesis algorithm designed: bearing comparison between consecutive path segments, turn classification thresholds, and floor-change handling	Nakshatra Agarwal
Class hierarchy finalised: Person, Patient and Staff with Doctor, Nurse and Admin, and the Charge hierarchy with a virtual cost calculation	Devesh Saini
Billing model specified: automatic accrual of bed, medicine, test and procedure charges, operator overloading for bill accumulation, and insurance deduction rules	Devesh Saini
Record schema and file formats defined, with the load-and-rebuild strategy for the hash index and the AVL date index	Vasav Sharma
Test plan written: unit tests per structure, scenario tests with known expected treatment order, and hand-calculated billing checks	Vasav Sharma
Updated system architecture, module boundaries and the mapping of every syllabus topic to a specific module	All members
Phase-I proposal report and evaluation deck completed and submitted	All members

Challenges/Roadblocks

Turning a shortest path into directions a person can follow has been the hardest part of this phase. Dijkstra's algorithm returns a list of vertex identifiers, and that list is of no use to an attendant pushing a trolley. Our fix was to put plan coordinates and a floor number on every vertex, work out the bearing of each segment, and classify the change in bearing from one segment to the next as a left turn, a right turn, a slight turn or straight ahead. Lifts and stairways are treated as a separate case and produce an instruction to change floor.

A second problem sits right next to it. The nearest free bed is often not the correct one, because a general ward bed may be much closer than the intensive care bed the patient actually needs. We now filter candidates by clinical type before any distance is measured, so a route is only ever computed towards a destination that is medically valid.

Making the C engine and the C++ object layer work together has taken care as well. Structures allocated in C and used from C++ need clear ownership rules, otherwise the same block is freed twice, and the C headers have to be wrapped so that they can be included from C++ translation units.

The AVL tree meets its own worst case here, since admission dates arrive in ascending order and would flatten an unbalanced tree into a linked list. Rotations are therefore essential rather than optional.

One further risk is organisational. Three people editing one codebase will eventually conflict, so each member owns separate modules and integration happens through reviewed merges rather than direct commits to the main branch.

Tasks Pending

Week	Task Pending	Team Member
Week 1 30 Sep – 5 Oct	Core data-structure engine in C: stack, queue, priority queue, circular queue and double-ended queue, each with its own unit tests	Nakshatra Agarwal
Week 2 6 Oct – 12 Oct	Object-oriented layer in C++: Person, Patient and Staff hierarchy, patient registration with unique identifier generation, and the hash index for lookup by identifier	Devesh Saini
Week 3 13 Oct – 19 Oct	Emergency triage on the priority queue, the hospital graph, and Dijkstra-based selection of the nearest free intensive care bed, operation theatre or ward	Nakshatra Agarwal
Week 4 20 Oct – 26 Oct	In-hospital navigation: turn-by-turn direction synthesis with lift and stairway handling, circular-queue scheduling of operation theatre slots, and double-ended queue handling of ambulance and transfer requests	Nakshatra Agarwal Vasav Sharma
Week 5 27 Oct – 1 Nov	Medical records on a doubly linked list with AVL date-range search, the billing engine with operator overloading and insurance deduction, file persistence, end-to-end integration and final testing	Devesh Saini Vasav Sharma

Project Outcome/Deliverables

The following are the key outcomes expected at the end of the project:

- a working console-based hospital simulator covering patient registration, severity-based emergency triage, automated routing, medical records and billing
- an in-hospital navigation feature that prints turn-by-turn walking directions from the emergency room to the assigned bed, including lift and stairway instructions across floors
- a persistent file-based storage layer, to ensure that patient, staff and billing records survive between program runs
- complete and documented source code in C and C++ that covers every core topic from the Data Structures and Object-Oriented Programming syllabi, organised into independently testable modules
- a tested billing engine that accumulates charges automatically as resources are consumed and deducts insurance before producing the final payable amount
- project documentation, consisting of this report, the updated architecture diagram, and a table mapping each feature to the data structure or object-oriented concept behind it

Progress Overview

Design is complete and the project is now well into implementation. Requirements are frozen, the architecture has been updated for the navigation module, the class hierarchy and record formats are specified, and the test plan is written and partly executed.

The core data-structure engine is the part that is ahead. Every structure it contains has been unit tested and all four of those tests pass, so triage ordering, operation theatre slot reuse, AVL balance under ascending dates and hash lookup under forced collisions all behave as specified.

Resource routing is the part that is behind. Our first scenario test failed: treatment order came out correct, but a patient needing intensive care was sent to a nearer general ward bed, because the search compared distance before it filtered candidates by clinical type. The fix is understood and is scheduled inside Week 3, and the same test will be rerun as the acceptance check.

Navigation, billing and persistence have not been exercised yet. The remaining work is divided into five weekly blocks ending on the first of November, with each member owning separate modules so that the three of us work in parallel. Integration and final testing keep a week of their own, and the graphical interface stays in Phase III so that it cannot crowd out core functionality.

Codebase Information

Repository: `https://github.com/naksh5116/Team-ID-DSCPP-III-2026-T120`

Branches: `main` for integrated code, with `feature/engine`, `feature/triage`, `feature/navigation` and `feature/billing` for module work. Each member commits from their own account on their own feature branch, and integration happens through reviewed merges into `main`.

Layout: `/src/engine` holds the C data-structure engine, `/src/model` the C++ class layer, `/src/modules` triage, navigation and billing, `/data` the record files and the hospital graph definition, and `/tests` the unit and scenario tests.

Milestone commits:

- `feat(engine): priority queue keyed on severity with arrival-time tie-break`
- `feat(engine): circular queue for OT slots and deque for transfer requests`
- `test(engine): unit tests for the queue family, AVL rotations and hash collisions`
- `feat(model): Person, Patient and Staff hierarchy with role behaviour`
- `feat(records): hash index for constant-time lookup by patient identifier`
- `feat(graph): hospital layout loader with floors, coordinates and edge types`
- `feat(routing): Dijkstra selection of the nearest free bed`
- `fix(routing): filter candidates by clinical type before comparing distance`
- `feat(nav): turn-by-turn direction generator with lift and stairway steps`
- `feat(billing): Charge hierarchy, operator+= accumulation, insurance deduction`
- `feat(persistence): fstream record writer and index rebuild on startup`

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Unit test: priority queue ordering	**Pass**	Patients inserted out of order are extracted in descending severity, with arrival time breaking ties correctly
Unit test: circular queue wrap-around	**Pass**	A freed operation theatre slot is reused immediately without shifting the remaining slots
Unit test: AVL balance on sorted input	**Pass**	Admission dates inserted in ascending order keep the tree height logarithmic; all four rotation cases exercised
Unit test: hash lookup and collisions	**Pass**	Records remain retrievable by identifier after collisions are forced into the same bucket
Scenario test: triage and routing	**Fail**	Treatment order is correct, but bed allocation is not. A patient needing intensive care was assigned a closer general ward bed, as a distance comparison was done before the clinical type was filtered. Fix scheduled for Week 3
Scenario test: navigation directions		Expected sequence of turns and floor changes for a known start and destination. Not yet executed; scheduled for Week 4
Calculation test: billing with insurance		Computed totals must match hand-calculated figures including the insurance deduction. Scheduled for Week 5
Integration test: full patient journey		Registration through triage, routing, consultation and billing to discharge, with records persisted and reloaded. Scheduled for Week 5

Deliverables Progress

- **Requirements and feature freeze** — Completed. All Phase-II features, including in-hospital navigation, are finalised.
- **Updated system architecture** — Completed. The four-module application layer and the extended graph model are specified.
- **Core data-structure engine in C** — Completed. All four unit tests pass.
- **Object-oriented class layer in C++** — In progress. Hierarchy in place; registration and identifier generation under development.
- **Emergency triage** — Completed. Severity ordering verified by scenario test.
- **Resource routing** — In progress. Scenario test currently failing on bed selection; fix scheduled inside Week 3.
- **In-hospital navigation module** — Pending. Design complete, implementation scheduled for Week 4.
- **Medical records with hashing and AVL indexing** — In progress. Indexes built and tested; visit history pending.
- **Billing engine with insurance deduction** — Pending. Scheduled for Week 5.
- **File-based persistence layer** — Pending. Scheduled for Week 5.
- **Testing and validation** — In progress. Four unit tests passed, one scenario test failed, three tests not yet run.
- **Qt graphical interface** — Deferred to Phase III, as planned.